ARCHON — Implementation Plan

Context

`D:\Archon` currently contains only `ARCHON.pdf` — a 70-section "Ultimate Master Engineering Prompt". It specifies a **greenfield** platform, **ARCHON: AI Software Archaeologist & Self-Healing Legacy Code Platform**: given an unfamiliar Python Git/GitHub repository, ARCHON builds an *evidence-backed* understanding of the code, scores its risk, generates tests, executes them in a sandbox, investigates failures, proposes and verifies minimal patches, remembers incidents, and recommends a safe modernization order.

The spec's non-negotiable core is one **closed loop**:

```
GitHub repo -> validate -> secure clone -> snapshot -> source analysis -> dependency analysis
-> git history -> architecture reconstruction -> legacy risk -> change safety -> test analysis
-> characterization / test generation -> secure test execution -> failure detection
-> AI root-cause investigation -> candidate patch generation -> sandbox patch application
-> patch verification -> regression verification -> VERIFIED / REJECTED -> incident memory -> modernization
```

Governing principles (from the spec): evidence-driven (deterministic analysis first, AI only where reasoning adds value); every AI conclusion carries *Conclusion + Confidence + Evidence + Source Location + Classification* (FACT / INFERENCE / HYPOTHESIS / RECOMMENDATION); every score has an *explicit, versioned, tested* algorithm; AI never invents repository facts (missing evidence → `UNKNOWN` / `LOW_CONFIDENCE`); untrusted repo/AI code never runs on the host; the original repository is never modified automatically; no fake data, no stubs presented as complete.

Decisions locked with the user:

- Plan covers the **full platform, all 12 phases**, plus full React frontend and Excel reporting.
- **Docker Desktop is available** — the sandbox is Docker-based.
- AI: build the **`AIProvider` abstraction + a deterministic mock/fixture provider first** so the whole loop runs offline; add a real Claude driver later behind the same interface.
- **Full frontend + Excel** are in scope (built incrementally, one screen/sheet per backend phase).

Outcome: a running system where `docker compose up` + a POST of a real GitHub URL (or `archon analyze <path>`) drives the entire loop and produces real artifacts, real scores with explanations, a real verified patch, and a regression check — reproducible via an end-to-end acceptance test on a purpose-built fixture repository.

Repository layout

```
D:\Archon\
  backend/
    archon/
      config/            settings (pydantic-settings), env profiles
      core/              structured logging, error taxonomy, ids, engine-version registry
      db/                SQLAlchemy models, session, migrations wiring
      domain/            pydantic domain models, Evidence model, classification enums, ai_schemas
      providers/
        repo/            RepositoryProvider ABC, LocalRepositoryProvider, GitHubRepositoryProvider
        ai/              AIProvider ABC, MockAIProvider, (later) ClaudeAIProvider
      workspace/          WorkspaceManager (temp dirs, quotas, guaranteed cleanup)
      jobs/              JobManager, StateMachine, DB-backed queue, worker entrypoint
      pipeline/          orchestrator that walks the state machine + persists checkpoints
      analysis/
        source/ git/ graph/ architecture/ archaeology/ legacy/ understanding/
        change_safety/ change_impact/
      scoring/           versioned scoring engines + registry (legacy_risk, change_safety, hotspot, understanding, p
      testing/           discovery/ coverage/ gaps/ characterization/ generation/
      sandbox/           Sandbox ABC, DockerSandbox, ExecutionSpec/Result, container reaper
      execution/         run suites via sandbox, capture Execution rows + coverage artifacts
      failure/           failure detection, stack-trace parsing, reproducibility
      investigation/     root-cause context assembly + AI RootCauseAnalysis
      healing/           patch generation (AI), static validation, apply, ranking
      verification/      patch verification gate + regression verification + rollback
      incidents/         incident memory store + similarity retrieval
```

```
    comparison/        repo@A vs repo@B diff
    modernization/     strategy generation + safe ordering
    reporting/         openpyxl workbook + repositories.xlsx bulk input
    api/               FastAPI routers (one per resource), error handlers, OpenAPI
    cli/               `archon` CLI (typer) for headless end-to-end runs
  tests/
    unit/ integration/ acceptance/
    fixtures/
      test_repo/       purpose-built git repo (built by a script that makes real commits)
      scoring/         synthetic components for metric property tests
      malicious/       hostile repo for sandbox containment tests
      ai/              canned MockAIProvider responses
  alembic/
  pyproject.toml       ruff, mypy, pytest, coverage config
frontend/              React + TypeScript + Vite
docker/
  Dockerfile.api  Dockerfile.worker  Dockerfile.sandbox  docker-compose.yml
docs/architecture/     the Phase 0 contract documents (below)
docs/roadmap.md
Makefile               dev / test / migrate / e2e targets
.env.example
```

Tech stack (from spec §18, no additions): Python 3.12+, FastAPI, Pydantic v2, SQLAlchemy 2.x, Alembic; PostgreSQL (prod) + SQLite (dev/test); Python `ast` (+ tree-sitter only if a concrete need appears); Git CLI + GitPython where useful; NetworkX (no Neo4j); pytest + coverage.py; Docker sandbox; React + TypeScript + Vite; openpyxl. AI via `AIProvider` abstraction (Claude / OpenAI / local behind one interface).

Phase 0 — Greenfield architecture & engineering contract

Deliver `docs/architecture/*.md` **and** the skeleton code they describe. These documents are the source of truth the spec demands (§8, §9, §60 "no undefined metrics").

Doc	Contents
`00-overview.md`	System architecture, component map, the evidence pipeline, agent/orchestration roles (§66 — logical roles over shared knowledge, not autonomous modules).
`01-domain-model.md`	Domain entities, the `Evidence` model, `Classification` enum, `Confidence` handling.
`02-database-schema.md`	Full schema (below), relationships, versioning, traceability, auditability.
`03-state-machine.md`	Analysis state machine (below): states, allowed transitions, per-state entry/exit/failure/retry/checkpoint/idempotency.
`04-artifact-storage.md`	What lives in DB vs filesystem vs object-storage abstraction vs logs vs temp workspace; reproducibility + cleanup rules.
`05-sandbox-threat-model.md`	Untrusted-code threat model + Docker isolation/network/CPU/memory/timeout/fs/env/process/cleanup policy (below).
`06-ai-output-schemas.md`	Versioned Pydantic schemas for every AI operation + the validation pipeline + hallucination controls.
`07-job-concurrency-model.md`	HTTP request vs analysis job vs background execution vs sandbox exec vs AI request; job id/status/retry/cancel/concurrency/dedupe/idempotency/progress/recovery.
`08-repository-limits.md`	Explicit initial limits + reject-vs-degrade behavior (below).
`09-supported-repository-contract.md`	SUPPORTED / PARTIALLY SUPPORTED / UNSUPPORTED conditions (below).
`10-scoring-models.md`	The five scoring engines fully specified (below).
`11-testing-architecture.md`	Test tiers, fixtures, per-phase completion gate, the E2E acceptance test.
`12-deployment.md`	docker-compose topology, worker/Docker access model, secret handling, Make targets.

Phase 0 code: `config`, `core` (logging, error taxonomy with *Error Code / Message / Context / Recoverability / Suggested Action* per §54, id + engine-version registry), all SQLAlchemy models + first Alembic migration, `domain` pydantic models + `Evidence` + enums, `ai_schemas` module (schemas only).

Phase 0 gate: `alembic upgrade head` green on SQLite **and** Postgres; model round-trip tests pass; all `docs/architecture/*.md` written; `ruff` + `mypy` clean.

Database schema (§9.1)

One table per spec entity, plus `Job`. Every analysis-output row carries `run_id`, `engine_version`, and `evidence_ids`; snapshots are immutable; runs reference exactly one snapshot; `AnalysisRun.config_hash` is the cache key (§53).

- **Repository**(id, provider, url, default_branch, limits_profile, created_at)
- **RepositorySnapshot**(id, repo_id, commit_sha, branch, workspace_ref, size_bytes, file_count, created_at) — immutable
- **AnalysisRun**(id, snapshot_id, mode, engine_versions JSON, state, current_stage, last_completed_stage, config_hash, started_at, ended_at, error JSON)
- **AnalysisArtifact**(id, run_id, kind, storage[db|fs|object], ref, sha256, size, mime, created_at) — large/generated blobs go to fs/object, never inflate rows (§11)
- **Component**(id, snapshot_id, kind[file|module|class|function|method], name, qualified_name, path, start_line, end_line, metrics JSON, role)
- **Dependency**(id, snapshot_id, src_component_id, dst_component_id, kind[IMPORTS|CALLS|INHERITS|DEPENDS_ON], evidence_id)
- **Commit**(id, repo_id, sha, author, authored_at, message, additions, deletions, changed_paths JSON)
- **Evidence**(id, run_id, classification[FACT|INFERENCE|HYPOTHESIS|RECOMMENDATION], summary, detail, source_path, source_line, confidence, produced_by, refs JSON) — central; every AI conclusion links here (§4)
- **RiskAssessment**(id, run_id, component_id, engine_version, score, category, factor_breakdown JSON, confidence, evidence_ids JSON)
- **LegacyDNA**(id, run_id, component_id, age_days, complexity, churn, coupling, coverage, failure_count, assumption_count, debt_score, legacy_risk_score, confidence, evidence_ids)
- **TechnicalDebtFinding**(id, run_id, component_id, category, location, severity, impact, confidence, recommendation, evidence_id)
- **Hotspot**(id, run_id, component_id, score, classification[STABLE|WATCH|RISKY|CRITICAL], reasons JSON, evidence_ids, engine_version)
- **Assumption**(id, run_id, component_id, kind, location, description, risk, confidence, suggested_test, evidence_id)
- **ChangeAssessment**(id, run_id, component_id, safety_score, risk_category, factor_breakdown JSON, recommended_preparation JSON, engine_version, evidence_ids)
- **ChangeImpact**(id, run_id, component_id, direct_dependents JSON, indirect_dependents JSON, callers JSON, related_tests JSON, historical_co_changes JSON, external_integrations JSON, potential_impact JSON)
- **TestCase**(id, run_id, component_id, kind[existing|characterization|unit|boundary|invalid_input|exception|regression|integration], path, name, body_ref, origin[discovered|ai|characterization], validated, validation_errors JSON)
- **Characterization**(id, run_id, component_id, input_spec JSON, observed_output_ref, observed_side_effects JSON, baseline_hash, test_case_id)
- **Execution**(id, run_id, kind, sandbox_ref, command, exit_code, passed, failed, errors, duration_ms, stdout_ref, stderr_ref, coverage_ref, started_at, ended_at)
- **Failure**(id, execution_id, test_identifier, message, exception_type, stack_trace_ref, parsed_frames JSON, reproducible, occurrences, first_seen)
- **Investigation**(id, failure_id, run_id, summary, root_cause_hypotheses JSON[{statement, confidence, evidence_ids}], affected_component_ids JSON, recommended_verification JSON, ai_schema_version)
- **Patch**(id, investigation_id, run_id, strategy, diff_ref, target_component_ids JSON, lines_added, lines_removed, static_validation JSON, rank_score, rank_breakdown JSON,

state[PROPOSED|TESTING|PARTIALLY_VERIFIED|VERIFIED|REJECTED], ai_schema_version)

- **PatchVerification**(id, patch_id, original_failure_fixed, characterization_pass, regression_pass, existing_tests_pass, new_critical_failures, applies_cleanly, verdict, execution_ids JSON, evidence_ids)
- **Incident**(id, repo_id, failure_signature, failure_summary, root_cause, evidence_ids, affected_component_ids JSON, fix_ref, patch_id, regression_test_ids JSON, verification_id, confidence, created_at)
- **ModernizationRecommendation**(id, run_id, target, strategy[add_tests|extract_dependency|refactor|replace_dependency|rewrite], risk, effort, impact, change_safety_ref, dependencies JSON, required_tests JSON, prerequisites JSON, order_index, rationale, evidence_ids)
- **Job**(id, run_id, type, state, priority, attempts, max_attempts, idempotency_key, dedupe_key, progress_pct, current_stage, heartbeat_at, error JSON, created_at, started_at, finished_at)

Analysis state machine (§10)

```
PENDING -> QUEUED -> INGESTING -> SNAPSHOTTING
-> ANALYZING_SOURCE -> ANALYZING_GIT -> BUILDING_GRAPH -> RECONSTRUCTING_ARCHITECTURE
-> ARCHAEOLOGIZING -> SCORING_UNDERSTANDING -> BUILDING_LEGACY_DNA -> ANALYZING_TECH_DEBT
-> SCORING_HOTSPOTS -> ASSESSING_CHANGE_SAFETY -> ANALYZING_CHANGE_IMPACT
-> ANALYZING_TESTS -> CHARACTERIZING -> GENERATING_TESTS -> EXECUTING -> DETECTING_FAILURES
-> INVESTIGATING -> GENERATING_PATCH -> RANKING_PATCHES -> VERIFYING_PATCH -> REGRESSION_VERIFYING
-> RECORDING_INCIDENT -> [MODERNIZING] -> COMPLETED
terminal: FAILED, CANCELLED
```

`StateMachine` class holds an explicit `ALLOWED: dict[State, set[State]]`; illegal transitions raise. Per state: entry preconditions, rows/artifacts produced, exit condition, failure mode (**analysis** stages *degrade-and-continue* recording a warning Evidence; **MVP-loop** stages *fail the run*), retry (only idempotent stages auto-retry with backoff), checkpoint (`last_completed_stage`), idempotency (stage re-entry = delete rows `WHERE run_id=? AND stage=?` then insert). No "magic status" values.

Sandbox threat model (§12) — Docker

All repo code, generated tests, and generated patches are **UNTRUSTED**. Static scan (no `os.system`/`subprocess`/`socket`/dangerous `__import__`, scope-limited diff) *flags* — never silently drops — before anything reaches the sandbox.

Per execution: one ephemeral container, non-root user, `--read-only` root fs with tmpfs `/work` + `/tmp`, `--cap-drop=ALL`, `--security-opt=no-new-privileges`, default seccomp, **`—network=none`** (opt-in `allow_install` uses a separate egress-filtered install phase; default fails clearly if deps missing), `--cpus`, `--memory` == `--memory-swap` (no swap), `--pids-limit`, ulimits (nofile/nproc), wall-clock kill by the orchestrator, empty env (never any ARCHON/Anthropic/GitHub secret), only the snapshot workspace copied in (no host mounts of ARCHON code), outputs copied out of `/work/out`, `--rm` + a startup/post-run reaper for orphaned containers and volumes. `Sandbox` ABC keeps a seam for a future non-Docker driver.

Repository limits (§16) — initial, env-overridable

max_repo_size 500 MB · max_file_size 2 MB · max_file_count 20 000 · max_git_history 5 000 commits · max_analysis_duration 30 min · max_generated_tests 200 · max_ai_context per provider window · max_patch_candidates 5 · max_sandbox_runtime 300 s/exec. Over a hard limit → **reject** with a structured reason; over a soft limit → **degrade** (cap history, skip AI test-gen, …) and record a warning Evidence. Never silent (§16, §54).

Supported-repository contract (§17)

- **SUPPORTED:** git repo, GitHub-hosted or local path, non-empty, has git history, primarily Python (≥ threshold of analyzed LOC), `ast`-parseable (per-file syntax errors tolerated with a recorded finding), dependency manifest present (`requirements.txt` / `pyproject.toml` / `setup.py`), pytest detectable → full loop incl. execution + self-healing.

- **PARTIALLY SUPPORTED:** Python, no tests (analysis + characterization + generation, no baseline pass rate) · no manifest (analysis only, best-effort execution) · shallow/no history (archaeology degraded) · mixed-language (Python analyzed, rest summarized).
- **UNSUPPORTED:** no Python, no git, exceeds hard limits, binary-only → rejected with reason. No multi-language claims beyond what is implemented.

Scoring models (§5–7, §27–31, §40, §60) — five versioned engines

Common shape: `scoring/` module per engine with `name`, `version` (e.g. `legacy_risk.v1`), input-signal spec, per-signal normalization to [0,1], **weights in a versioned config module**, formula, thresholds, output range, categories, evidence mapping, confidence handling, and `explain()` returning each factor's contribution. A `ScoringRegistry` records active versions onto `AnalysisRun.engine_versions`. Synthetic-fixture acceptance tests assert the monotonic properties from §7 (and document any deliberate exception).

1. **Legacy Risk** (per component, 0–100, LOW/MODERATE/HIGH/CRITICAL). Signals: complexity, churn, coverage-gap (`1-coverage`), coupling (fan-in+fan-out centrality), historical failures, hidden-assumption count, weighted tech-debt, age. Weighted sum with weights tuned so **churn + complexity + low coverage dominate** (§7: high-churn/high-complexity/low-coverage must rank riskier than a stable equivalent). Confidence = fraction of signals backed by real data vs defaulted.
2. **Change Safety** (per component, 0–100, higher = safer; SAFE/CAUTION/RISKY/DANGEROUS). Signals: coverage (+), historical change-success rate (+), low complexity, low coupling, low dependency centrality, few callers-at-risk, few assumptions. §7: a well-covered stable component must score safer than a highly-coupled historically-unstable one.
3. **Hotspot Score** (per component, 0–100, STABLE/WATCH/RISKY/CRITICAL). Combines normalized complexity/churn/coupling/coverage-gap/failures/assumptions/debt with a **multiplicative overlap bonus when ≥3 signals exceed threshold** (the "signals overlap" idea of §29).
4. **Repository Understanding** (0–100 + confidence + evidence coverage). Dimensions: architecture, dependency, behavior, historical, testing, configuration understanding — each derived from evidence coverage (% components with resolved roles, % imports resolved, % functions with behavior analysis, history depth available, coverage data present, configs parsed). **Sparse evidence lowers confidence** (§30).
5. **Patch Ranking** (orders candidates; the VERIFIED gate is separate, §41). Signals: original-failure-fixed (strong), relevant/regression/characterization pass rate, regression risk (heavy penalty for new failures), patch size (inverse), complexity delta, coupling delta, behavior-preservation (characterization pass rate), security-scan result. §7: a patch that passes more relevant verification **and** changes less must rank above a larger unverified patch. Never auto-select the first AI patch (§40).

AI structured-output contract (§13–14)

Versioned Pydantic schemas: `BehaviorAnalysis`, `HistoricalIntent`, `AssumptionAnalysis`, `RootCauseAnalysis`, `TestGeneration`, `PatchProposal`, `PatchRanking`, `ModernizationRecommendation`. Common envelope: `result`, `evidence: list[EvidenceRef]` (every repo-specific claim resolves to a real `Component`/`Commit`/file/`TestCase` row or is `UNKNOWN`), `confidence`, `reasoning_summary`, `affected_components`, `recommended_action`, `classification`. `AIProvider.complete_structured(prompt, schema, context) -> schema`. `MockAIProvider` returns deterministic fixture responses keyed by scenario (all tests + offline dev run through it); `ClaudeAIProvider` added later. Validation pipeline: JSON-schema → evidence validation (unresolvable claim → reject or downgrade to `LOW_CONFIDENCE`/`UNKNOWN`) → domain validation → execution validation for patches/tests. Malformed output → typed `AIOutputError`, never silently accepted.

Job / concurrency model (§15)

HTTP request creates `AnalysisRun` (PENDING) + `Job` (QUEUED) and returns the run id immediately. A separate `archon-worker` process (same codebase) consumes a **DB-backed queue** (`SELECT … FOR UPDATE SKIP LOCKED` on Postgres; polling on SQLite). Concurrency: global running-job cap, **one running run per `repo_id + config_hash`** (dedupe), sandbox-concurrency semaphore. Retry per idempotent stage with backoff. Cooperative cancellation checked between stages and inside long loops; sandbox execs killed.

`idempotency_key` header dedupes run creation. Progress = `progress_pct + current_stage + heartbeat_at`; stale heartbeat → requeue. Celery/RQ/Arq can slot behind `JobManager` later — not added now (§19).

Phases 1–12 — implementation roadmap

Priority order follows spec §65. Every phase runs the §62 workflow — **understand** → **design** → **implement** → **unit test** → **integrate** → **integration test** → **regression test** → **completion gate** — and each phase's **completion gate** is: unit + integration + acceptance tests green, wired into the pipeline **and** an API route **and** (from Phase 3 on) a frontend screen, with no stub/TODO left representing complete work.

> **Sequencing note:** the Docker `Sandbox` core (Phase 8 in spec numbering) is built **before** Phase 7's execution needs it — Phase 7's characterization and coverage steps depend on the sandbox. Order below reflects that.

Phase 1 — Foundation & repository ingestion (§20–21)

- `RepositoryProvider` ABC; `LocalRepositoryProvider`; `GitHubRepositoryProvider` — URL validation, metadata via GitHub REST (auth, rate limits, timeouts, retries, pagination, not-found, private, clone-failure), branch/commit resolution, secure clone via **git CLI with arg lists (never `shell=True`)**, depth policy from limits, token only from env, **no credentials in URLs or logs**.
- `WorkspaceManager` — temp workspace under a configured root, quota check, guaranteed cleanup, path-traversal-safe.
- `JobManager` + `StateMachine` + worker entrypoint + DB queue; `pipeline` orchestrator shell that walks states and persists checkpoints.
- API: `POST /repositories`, `POST /repositories/{id}/runs`, `GET /runs/{id}` (status + progress). CLI: `archon analyze <url|path> --branch …`.
- Frontend: Repository Management + run-status screen.
- **Acceptance:** ingest the fixture repo + one small public GitHub repo → `RepositorySnapshot` with commit SHA; error cases (not found / private / empty / over-limit) return structured errors.

Phase 2 — Source intelligence (§22)

- Python `ast` extractor: files, modules, classes, functions, methods, imports, inheritance, decorators, args, returns, raised exceptions, intra-repo call edges (best-effort resolution), cyclomatic complexity, LOC, entry points (`__main__`, console_scripts, framework mains), test-file + config-file detection. Preserve file/line/symbol on every `Component`.
- Persist `Component` + `Dependency`(IMPORTS/CALLS/INHERITS) + `Evidence`(FACT) per extraction.
- **Acceptance:** a known fixture file → exact expected components, edges, complexity numbers.

Phase 3 — Architecture & dependency intelligence (§23)

- NetworkX graph from `Component` + `Dependency`; node kinds and relationship kinds per spec (CONTAINS, IMPORTS, CALLS, INHERITS, DEPENDS_ON, TESTED_BY, CHANGED_BY, CHANGED_WITH, FAILED_IN, FIXED_BY, AFFECTS).
- Architecture reconstruction: module clustering; role inference (entrypoint / api / domain / io / util / test / config) from imports, paths, framework signals, configuration; callers/dependents; centrality metrics. Persist `role` on `Component`; save graph as `AnalysisArtifact` (JSON/GraphML on fs).
- API: `GET /runs/{id}/architecture`, `/dependencies`. Frontend: Architecture + Dependency Graph views.
- **Acceptance:** fixture graph node/edge shape and inferred roles match expectations.

Phase 4 — Software archaeology (§24–26)

- **Deterministic git analysis:** commits, messages, authors, dates, changed files, additions/deletions, churn, change frequency, component age, co-changing files (CHANGED_WITH), CHANGED_BY edges (GitPython or git CLI).

- **Behavior reconstruction:** deterministic assembly of purpose / inputs / outputs / side-effects / exceptions / callers / tests / likely invariants from source + graph + git; AI `BehaviorAnalysis` + `HistoricalIntent` (mock provider) for "Why does this exist?" with FACT/INFERENCE/HYPOTHESIS/RECOMMENDATION + evidence + confidence.
- **Hidden assumptions:** deterministic heuristics (null / empty-collection / initialization / database / environment / external-API / ordering / timezone / schema) + AI `AssumptionAnalysis` interpretation; persist `Assumption` rows with suggested tests.
- API: archaeology / evolution / behavior / assumptions. Frontend: Git Evolution, Software Archaeology, Why Does This Exist, Hidden Assumptions.
- **Acceptance:** planted co-change / churn / age values recovered; planted assumptions detected.

Phase 5 — Legacy DNA, technical debt, hotspots, understanding (§27–30)

- Implement the **Legacy Risk**, **Hotspot**, **Repository Understanding**, and tech-debt severity engines per `docs/architecture/10-scoring-models.md`.
- Tech-debt detectors: long functions, large classes, duplicate logic, circular dependencies, high coupling, low cohesion, dead-code candidates, deprecated APIs, hardcoded config, broad `except`, silent failures, global state, magic numbers → `TechnicalDebtFinding` (category / location / evidence / severity / impact / confidence / recommendation).
- Persist `LegacyDNA`, `Hotspot`, `RiskAssessment`, understanding score.
- API: legacy-dna / hotspots / technical-debt / understanding. Frontend: Legacy DNA, Hotspots, Technical Debt, Repository Understanding.
- **Acceptance (metric property fixtures):** churn+complexity+low-coverage ranks riskier than stable equivalent; stronger evidence → higher understanding + confidence; higher coverage does not raise risk without documented reasoning; documented exceptions tested for intended behavior.

Phase 6 — Change safety & change impact (§31–32)

- **Change Safety** engine (versioned, explainable) from callers, dependency centrality, coverage, historical changes, historical failures, complexity, hidden assumptions, coupling → score / category / factors / recommended preparation.
- **Change Impact** for a selected component: direct + indirect dependents, callers, related tests (TESTED_BY), historical co-changes, external integrations, "what could break / which tests to run / what to do first".
- API: `GET /runs/{id}/change-safety`, `POST /runs/{id}/change-impact`. Frontend: Change Safety, Change Impact.
- **Acceptance:** stable well-tested component scores safer than a coupled unstable one; impact set matches fixture dependency graph.

*Phase 7 — Secure execution / sandbox (§12, §36) *(built before the rest of characterization)**

- `DockerSandbox` implementing the threat-model doc; `ExecutionSpec` / `ExecutionResult`; offline default vs opt-in egress-filtered dependency-install phase; container reaper.
- `execution` engine runs suites through the sandbox (existing tests, characterization, generated tests, patch verification, regression) and persists `Execution` + coverage artifacts.
- **Acceptance:** the `fixtures/malicious/` repo (network call, fork-bomb attempt, secret read, fs-escape attempt) is contained and reported; a normal pytest suite runs and results are captured.

Phase 8 — Characterization & test-gap analysis (§33–35)

- Existing-test discovery + coverage analysis (pytest + coverage.py via the sandbox).
- Characterization workflow: target → safety analysis → bounded safe input generation (deterministic strategies + AI `TestGeneration` scenarios) → execute current behavior in sandbox → capture output + side effects → emit characterization test → store baseline (`Characterization` + `TestCase`). **Observed behavior is explicitly not assumed correct** (§33, Principle 14).

- Test-gap analysis: untested functions/branches, missing edge/exception/regression/characterization tests, **prioritized by Legacy Risk / Change Safety / historical failures**.
- AI test generation (mock): unit / boundary / invalid-input / exception / regression / integration; every generated test **static-validated + sandbox-validated** before it counts.
- API: characterization / tests / test-gaps / executions. Frontend: Characterization, Test Intelligence.
- **Acceptance:** the fixture's known test gap is found and prioritized; a characterization baseline is reproducible run-to-run.

Phase 9 — Failure investigation & self-healing (§37–43)

- **Failure detection** + stack-trace parsing (frames → file/line/function mapped to `Component`), reproducibility check (re-run N times), intermittent-failure classification.
- **Investigation:** assemble context (stack, source, graph, git, assumptions, Legacy DNA, characterization, similar incidents) → AI `RootCauseAnalysis` (mock) → `Investigation` rows with ranked hypotheses + evidence + confidence + recommended verification. Proceed to healing only past a documented evidence/confidence threshold (§38).
- **Patch generation:** AI `PatchProposal` (mock) under the **Minimal Patch Principle** (§39, Principle 12) → unified diff → static validation (parses, no banned constructs, scope-limited) → apply in a throwaway workspace → run tests in the sandbox.
- **Patch ranking** engine (versioned, explainable) per doc — never just the first candidate.
- **Patch verification** (§41): VERIFIED only when original failure fixed **and** characterization passes **and** regression passes **and** relevant existing tests pass **and** no new critical failure **and** applies cleanly. States PROPOSED / TESTING / PARTIALLY_VERIFIED / VERIFIED / REJECTED.
- **Rollback** (§42): on failure → reject, restore workspace, preserve evidence, try next candidate. **Original repository never modified** (Principle 11). Human-approval step surfaced via API/UI; patch exported as a `.diff` artifact, never auto-applied (§43).
- API: failures / investigations / patches / verification. Frontend: Failures, Root Cause Analysis, Self-Healing, Patch Verification.
- **Acceptance:** fixture's reproducible bug → investigation names the planted root cause → a ranked minimal patch is generated, verified, and regression-checked; a deliberately bad candidate is rejected and rolled back.

Phase 10 — Incident memory (§44)

- On a verified repair, write an `Incident` (failure signature hash, failure, root cause, evidence, affected components, commit/snapshot, patch, regression test, verification, confidence).
- Retrieval by failure-signature + stack + component overlap; surfaced into future investigations as **clearly-marked historical context that never overrides current evidence** (§44, Principle 15).
- API + Frontend: Incident Memory.
- **Acceptance:** two similar failures → the second investigation cites the first incident without overriding fresh evidence.

Phase 11 — Repository comparison (§45)

- Analyze at commit A and commit B (two snapshots/runs); diff architecture, dependencies, Legacy DNA, technical debt, risk, coverage, change safety. Report as artifact + API + UI.
- **Acceptance:** fixture at two commits → expected deltas (added module, risk change).

Phase 12 — Modernization (§46)

- Deterministic inputs (deprecated deps, legacy APIs, high coupling, weak testing, hardcoded config, error handling, global state, architecture problems, technical debt, legacy hotspots) → AI `ModernizationRecommendation` (mock): add tests / extract dependency / refactor / replace dependency / rewrite, each with risk / effort / impact / change-safety / dependencies / required tests / prerequisites. **No automatic preference for rewrites** (§46, Principle 12). Safest-order sequencing derived from the

dependency + change-safety graph.

- API + Frontend: Modernization.
- **Acceptance:** fixture → "add tests before refactor" ordering; rewrite not chosen when a cheaper safe option exists.

Cross-cutting (built alongside every phase, hardened at the end)

- **REST API (§47):** routers for every listed resource; Pydantic validation, pagination, filtering, structured errors, OpenAPI. Every AI conclusion in responses carries conclusion + confidence + evidence + source location + classification.
- **Frontend (§48):** React + TS + Vite; all listed areas; **data only from real backend APIs**, run-status polling, evidence/confidence/classification shown on every AI conclusion. One screen ships per backend phase.
- **Excel (§49–50):** `reporting/` with openpyxl → `ARCHON_Legacy_Analysis.xlsx` (Executive Summary, Repository Understanding, Architecture, Legacy DNA, Change Safety, Change Impact, Technical Debt, Test Gaps, Characterization, Failures, Repairs, Modernization, Software Archaeology, Incident Memory) — **sourced from the same service/domain layer as the API**, no separate engine. Bulk input `repositories.xlsx` (Repository URL, Branch, Analysis Mode, Priority) → validation → enqueues ordinary `AnalysisRunS`.
- **Observability (§55):** run/repo/snapshot/commit/mode/current-stage/status/start/end/duration/errors/AI-activity/execution-results tracked and exposed.
- **Security (§52):** subprocess arg-lists only (no `shell=True`), path-traversal guards, secret redaction in logs, untrusted-code handling on every repo/AI-code path; no AI keys / GitHub tokens / secrets ever exposed or passed to the sandbox.
- **Performance / caching (§53):** cache key = repo + commit SHA + analysis config + engine version; reuse snapshots / AST / graph; never reuse across incompatible snapshots or engine versions.
- **Optional GitHub webhook (§51):** only after the MVP loop is stable — signature validation, duplicate-event suppression, reuse of existing engines (changed components → change impact → change safety → targeted tests → failure detection).

Testing architecture (§56–57)

- Tiers under `backend/tests/`: `unit/`, `integration/`, `acceptance/`; fixtures under `tests/fixtures/`.
- **`fixtures/test_repo/`** — a hand-built git repository created by a script that makes **real commits**, containing: legacy code, multiple modules, real dependency relationships, real git history, existing tests, **one known test gap**, **one reproducible bug with a fixable root cause**.
- **End-to-end acceptance test** (`tests/acceptance/test_end_to_end.py`) drives the full pipeline on `test_repo`: ingestion → analysis → architecture → archaeology → risk → change safety → characterization → testing → failure → investigation → patch → verification → regression, asserting real artifacts/rows at each stage and a final **VERIFIED** patch with regression pass.
- **Metric acceptance fixtures** (`tests/acceptance/scoring/`) — synthetic components asserting the §7 monotonic properties for all five engines, with documented exceptions tested for intended behavior.
- **Sandbox containment test** — `fixtures/malicious/` proves isolation.
- Per-phase **completion gate** as defined above; no phase proceeds on a broken foundation (§64: STOP → DIAGNOSE → ROOT CAUSE → FIX → TEST → INTEGRATE → REGRESSION → CONTINUE).

Deployment (§18, Docker)

- `docker/docker-compose.yml`: `api` (uvicorn), `worker` (job consumer, with Docker access to launch sandbox containers), `db` (PostgreSQL). `Dockerfile.sandbox` is built and available to the worker; sandbox containers run with the locked-down flags above.

- `.env.example`: `DATABASE_URL`, `GITHUB_TOKEN` (optional, for private/rate-limited), `ANTHROPIC_API_KEY` (unused until the Claude driver lands). Secrets never baked into images and never passed into the sandbox.
- `Makefile`: `make dev` (compose up), `make migrate` (alembic), `make test` (unit + integration), `make e2e` (end-to-end acceptance), `make lint` (ruff + mypy).

Verification — how to prove ARCHON works end-to-end

1. `make migrate` succeeds on **SQLite and PostgreSQL**; model round-trip tests pass.
2. `make test` green; `pytest tests/acceptance/scoring` green (all five engines' property tests).
3. `pytest tests/acceptance/test_end_to_end.py` runs the **entire loop** on `fixtures/test_repo/` and asserts: snapshot with commit SHA, source/architecture/git/legacy-DNA/hotspot/change-safety rows populated, the known test gap flagged, the reproducible bug detected, an investigation naming the planted root cause, a ranked minimal patch reaching **VERIFIED**, regression pass, and an `Incident` recorded.
4. Sandbox containment test: `fixtures/malicious/` cannot reach the network, exhaust the host, read secrets, or escape the workspace — each attempt is contained and reported.
5. `docker compose up`, then `POST /repositories` + `POST /repositories/{id}/runs` with a real small public Python GitHub repo; poll `GET /runs/{id}` to `COMPLETED`; `GET` architecture / legacy-dna / hotspots / change-safety / failures / patches all return real, evidence-backed data with confidence + classification.
6. `GET /runs/{id}/report.xlsx` downloads `ARCHON_Legacy_Analysis.xlsx`; every sheet's numbers match the corresponding API responses.
7. Open the frontend; each screen renders the same data from the live APIs — no hardcoded values anywhere.
8. `archon analyze <path-to-test_repo>` completes the full loop headless from the CLI.
9. `repositories.xlsx` with 2–3 rows enqueues ordinary runs through the same pipeline (no separate engine).